



VPC Endpoints



Kiprono Yegon

vpce-00cdae87bb82079f3 / NextWork VPC Endpoint

Endpoint ID

vpce-00cdae87bb82079f3

VPC ID

vpce-07ac3e882c67e3d1 (NextWork-vpc)

DNS record IP type

service-defined

Private DNS specified domains

-

Status

Available

Status message

-

IP address type

ipv4

Creation time

Tuesday, 12 May 2026 at 15:51:05 GMT+3

Service name

com.amazonaws.us-west-2.s3

Service region

us-west-2

Endpoint type

Gateway

Private DNS names enabled

No

Private DNS preference

-



Introducing Today's Project!

What is Amazon VPC?

Amazon VPC is a service from Amazon Web Services that allows you to create a private, isolated network within the cloud where you can securely run your applications and resources. It is useful because it gives you full control over your networking environment, including IP address ranges, subnets, routing, and security settings, enabling you to protect your systems, manage traffic flow, and design scalable architectures.

How I used Amazon VPC in this project

In today's project, I used Amazon VPC to isolate the resources that I needed for the project and allow interdependence

One thing I didn't expect in this project was...

The project is very insightful on security of resources in the cloud

This project took me...

This project took me about one and a half hours



In the first part of my project...

Step 1 - Architecture set up

I will : Create a VPC from scratch! Launch an EC2 instance, which you'll connect to using EC2 Instance Connect later. Set up an S3 bucket.

Step 2 - Connect to EC2 instance

In this step, I will connect to the EC2 instance and try access S3 through the public internet!

Step 3 - Set up access keys

In this step, I will create access keys because they give the EC2 instance access to the AWS environment.

Step 4 - Interact with S3 bucket

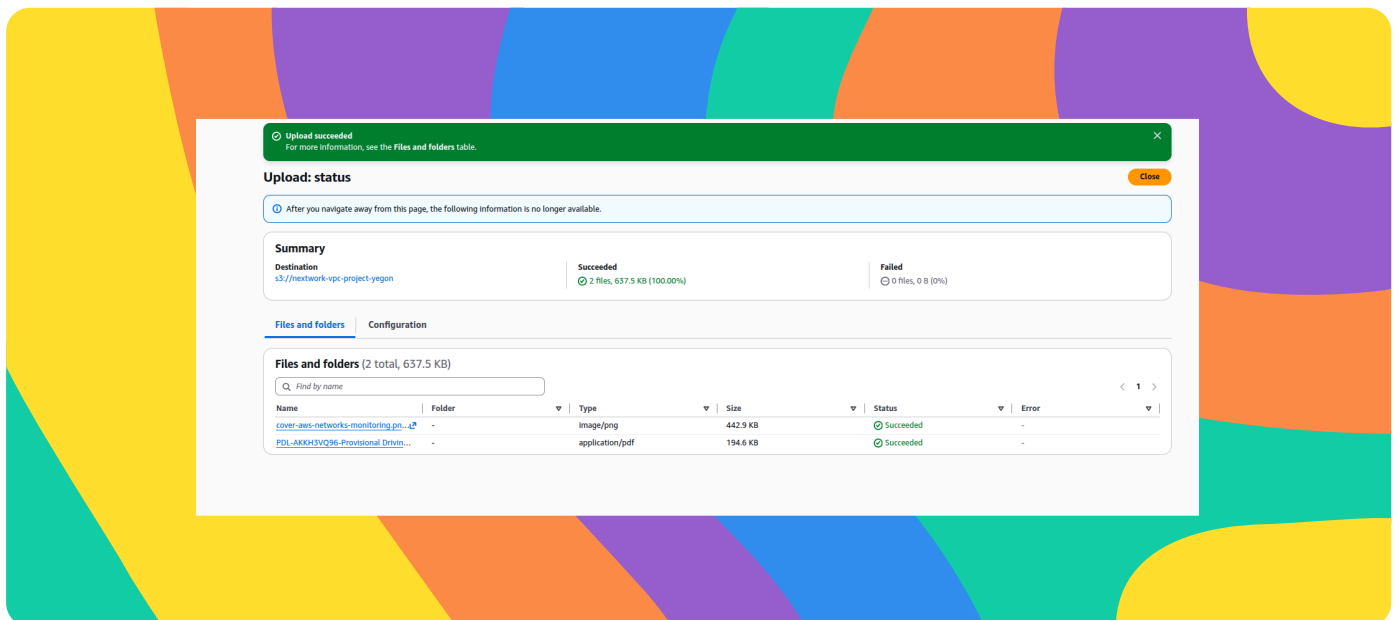
In this step, I will use the EC2 instance to access the S3 bucket .



Architecture set up

I started my project by launching an EC2 instance and enabled the automatic IP address to be able to use Instance Connect

I also set up an S3 bucket and uploaded two files





Access keys

Credentials

To set up my EC2 instance to interact with my AWS environment, I configured the Access Key Id, Secret Access Key, Default Region Name and the default output format which is left at the default

Access keys are credentials for applications and other servers to log into AWS and talk to your AWS services/resources.

Secret access keys are like passwords that pair with access key ID and is needed for AWS services' access

Best practice

Although I'm using access keys in this project, a best practice alternative is to use IAM role with the necessary permissions and then attaching that role to your EC2 instance.



nextwork.org

The command I ran was "aws s3 ls". This command is used to list all the S3 buckets in the Amazon s3 storage

The terminal responded with a list of the S3 buckets available in the account. This indicated that the access keys I set up connected my instance to Amazon services.

```

#
#####
Amazon Linux 2023
#####
|#####|
|#####|
|#####|
X--V- -> https://aws.amazon.com/linux/amazon-linux-2023
m/

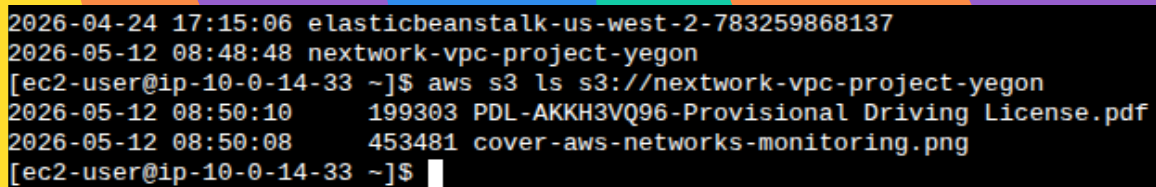
ec2-user@ip-10-0-14-33 ~]$ aws s3 ls

Unable to locate credentials. You can configure credentials by running "aws login".
ec2-user@ip-10-0-14-33 ~]$ aws configure
AWS Access Key ID [None]: AWS Secret Access Key [None]: Default region name [None]: Default output format [None]: [ec2-user@ip-10-0-14-33 ~]$ aws configure
AWS Access Key ID [None]: AKIA3MXPNDPU2JGYQZSU
AWS Secret Access Key [None]: 3JboKsqURGmrQt/rm4btmIW8tHjCTOsPtAqEdRs
default region name [None]: us-west-2
default output format [None]: [ec2-user@ip-10-0-14-33 ~]$ aws s3 ls
2026-04-17 17:59:21 elasticbeanstalk-eu-north-1-783259868137
2026-04-24 17:15:06 elasticbeanstalk-us-west-2-783259868137
2026-05-12 08:48:48 nextwork-vpc-project-yegon
ec2-user@ip-10-0-14-33 ~]$
```



Connecting to my S3 bucket

I also tested the command `aws s3 ls s3://nextwork-vpc-project-yegon` which returned a list of the files I had uploaded to the bucket



```
2026-04-24 17:15:06 elasticbeanstalk-us-west-2-783259868137
2026-05-12 08:48:48 nextwork-vpc-project-yegon
[ec2-user@ip-10-0-14-33 ~]$ aws s3 ls s3://nextwork-vpc-project-yegon
2026-05-12 08:50:10      199303 PDL-AKKH3VQ96-Provisional Driving License.pdf
2026-05-12 08:50:08    453481 cover-aws-networks-monitoring.png
[ec2-user@ip-10-0-14-33 ~]$
```



Uploading objects to S3

To upload a new file to my bucket, I first ran the command "sudo touch /tmp/test.txt". This command creates a file "test.txt" in the tmp folder

The second command I ran was "aws s3 cp /tmp/test.txt s3://nextwork-vpc-project-yegon". This command will copy the test.txt file to the s3 bucket

The third command I ran was... which validated that... The third command I ran was "aws s3 ls s3://nextwork-vpc-project-yegon" which validated that the test.txt file that was created earlier has been uploaded to the S3 bucket

```
[ec2-user@ip-10-0-14-33 ~]$ aws s3 ls
Unable to locate credentials. You can configure credentials by running "aws login".
[ec2-user@ip-10-0-14-33 ~]$ aws configure
AWS Access Key ID [None]: AKIA3MXPNDPU2J6VQZ5U
AWS Secret Access Key [None]: 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
Default region name [None]: us-west-2
Default output format [None]: [ec2-user@ip-10-0-14-33 ~]$ aws s3 ls
2026-04-17 18:59:21 elasticbeanstalk-us-north-1-783259868137
2026-04-24 17:15:06 elasticbeanstalk-us-west-2-783259868137
2026-05-12 08:48:48 nextwork-vpc-project-yegon
[ec2-user@ip-10-0-14-33 ~]$ aws s3 ls s3://nextwork-vpc-project-yegon
2026-05-12 08:50:10 199383 PDL-AKKH3VQ96-Provisional Driving License.pdf
2026-05-12 08:50:08 453481 cover-aws-networks-monitoring.png
[ec2-user@ip-10-0-14-33 ~]$ aws s3 cp /tmp/test.txt s3://nextwork-vpc-project-yegon
The user-provided path /tmp/test.txt does not exist.
[ec2-user@ip-10-0-14-33 ~]$ sudo touch /tmp/test.txt
sudo: touch: command not found
[ec2-user@ip-10-0-14-33 ~]$ sudo touch /tmp/test.txt
[ec2-user@ip-10-0-14-33 ~]$ aws s3 cp /tmp/test.txt s3://nextwork-vpc-project-yegon
upload: ../../tmp/test.txt to s3://nextwork-vpc-project-yegon/test.txt
[ec2-user@ip-10-0-14-33 ~]$ aws s3 ls s3://nextwork-vpc-project-yegon
2026-05-12 08:50:10 199383 PDL-AKKH3VQ96-Provisional Driving License.pdf
2026-05-12 08:50:08 453481 cover-aws-networks-monitoring.png
2026-05-12 08:04:45 0 test.txt
[ec2-user@ip-10-0-14-33 ~]$
```




In the second part of my project...

Step 5 - Set up a Gateway

In this step, I will set up a way for the VPC and S3 to communicate directly because the EC2 instance is connected to the bucket through the public internet. This isn't the most secure way to communicate - external threats and attacks can easily intercept your commands and get access to your AWS environment or sensitive data.

Step 6 - Bucket policies

We will validate the endpoint connection by blocking the S3 bucket from ALL traffic except the traffic coming from the endpoint and test if the instance can still access the S3.

Step 7 - Update route tables

In this step, I will access the S3 via the EC2 instance to check the connectivity via the VPC Endpoint and troubleshoot any arising error

Step 8 - Validate endpoint connection

In this step, I will test the VPC endpoint set up. Restrict your VPC's access to your AWS environment.

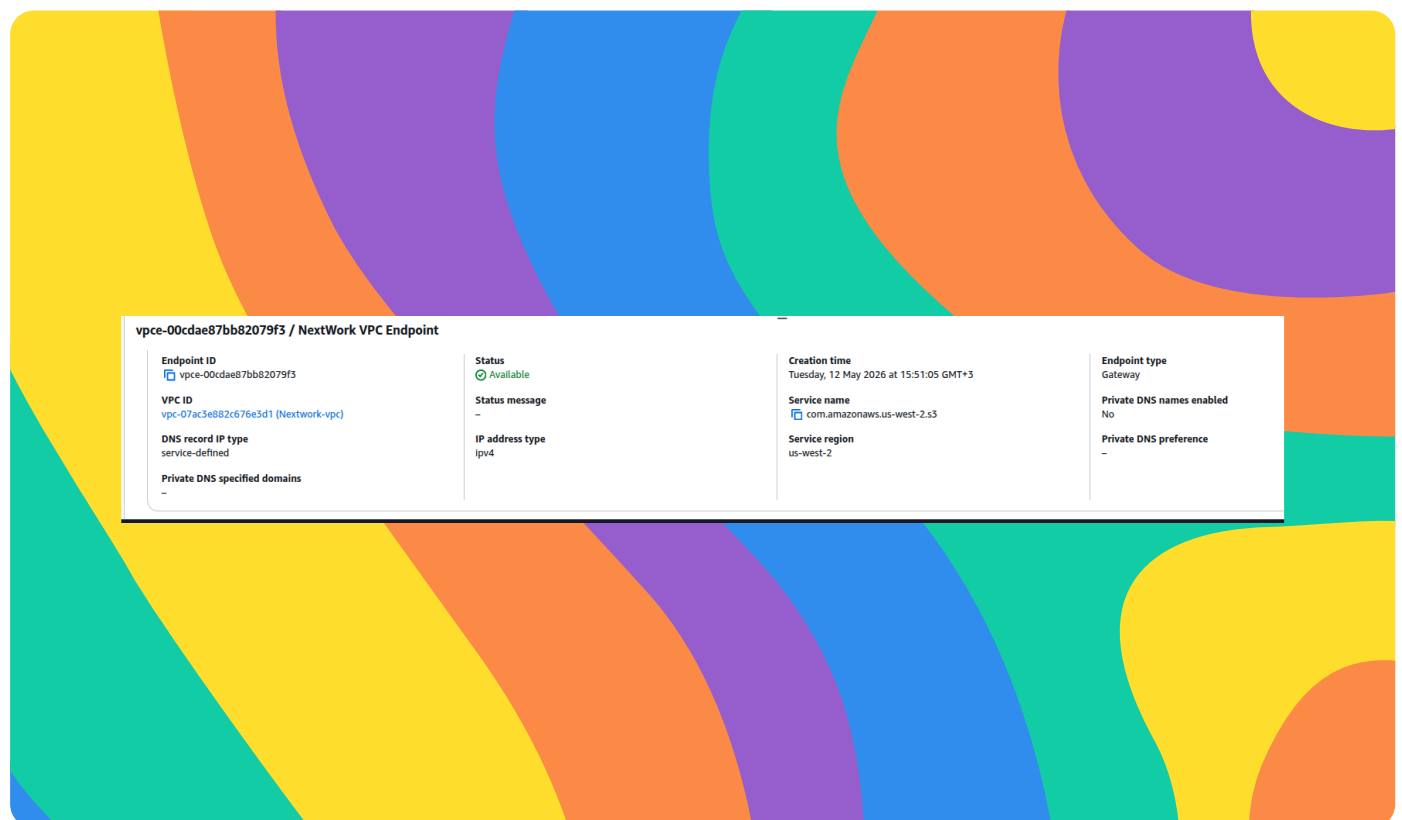


Setting up a Gateway

I set up an S3 Gateway, which is an endpoint used specifically for Amazon S3 and DynamoDB (DynamoDB is an AWS database service).

What are endpoints?

An endpoint is a service that allows private connections between your VPC and other AWS services without needing the traffic to go over the internet.





Bucket policies

A bucket policy is a type of IAM policy designed for setting access permissions to an S3 bucket. Using bucket policies, you get to decide who can access the bucket and what actions they can perform with it.

My bucket policy will deny all actions on your S3 bucket and its objects to everyone unless the access is from the VPC endpoint

Policy

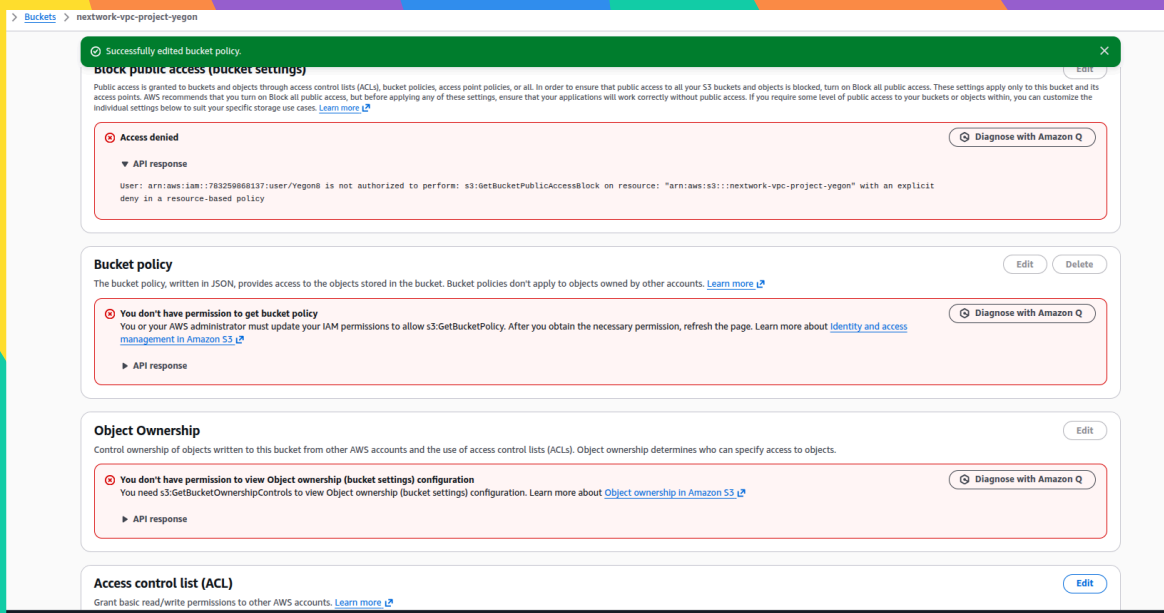
```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Deny",
6       "Principal": "*",
7       "Action": "s3:*",
8       "Resource": [
9         "arn:aws:s3:::nextwork-vpc-project-yegon",
10        "arn:aws:s3:::nextwork-vpc-project-yegon/*"
11      ],
12      "Condition": {
13        "StringNotEquals": {
14          "aws:sourceVpc": "vpce-00cdae87bb82079f3"
15        }
16      }
17    }
18  ]
19 }
20
21
```



Bucket policies

Right after saving my bucket policy, my S3 bucket page showed 'denied access' warnings. This was because the policy denies all actions unless they come from the VPC endpoint. This means any attempt to access the bucket from other sources, including the AWS Management Console, is blocked!

I also had to update my route table because it gives the route that directs traffic bound for S3 to your VPC endpoint, traffic from your EC2 instance is actually trying to get to your S3 bucket through the public internet instead

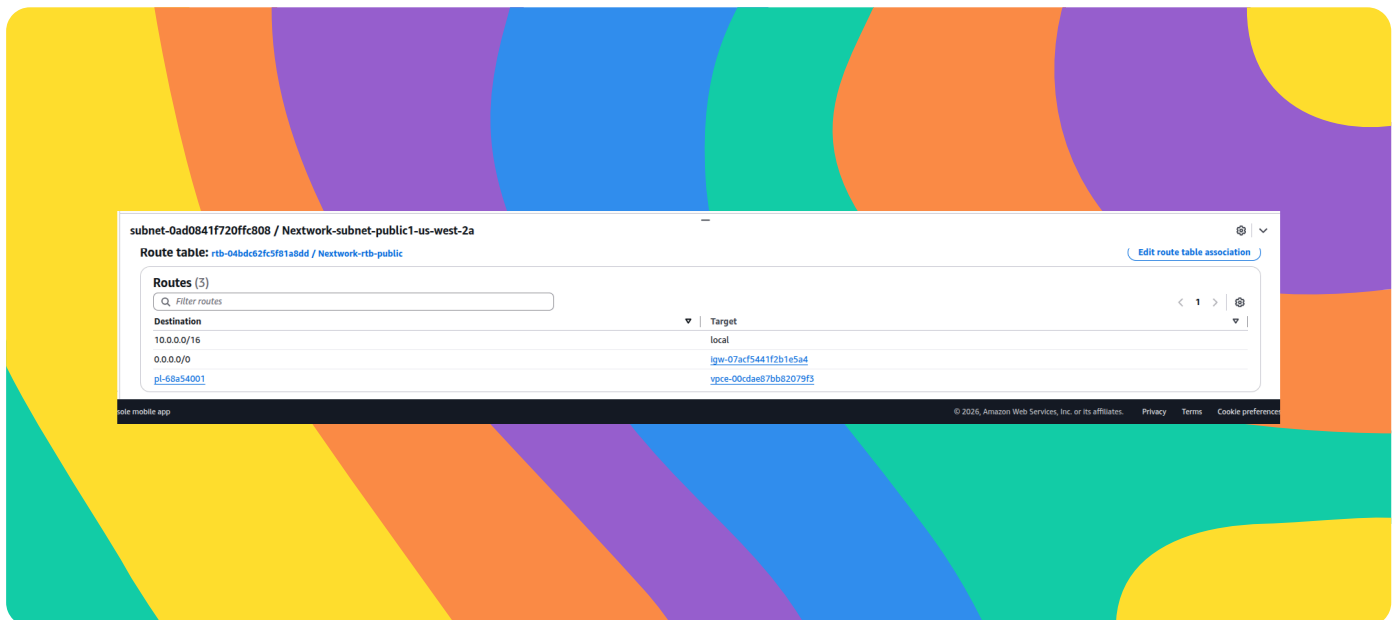




Route table updates

To update my route table, I modified the route table allowing the subnet in the route table

After updating my public subnet's route table, my terminal could return the list of the files in the S3 bucket

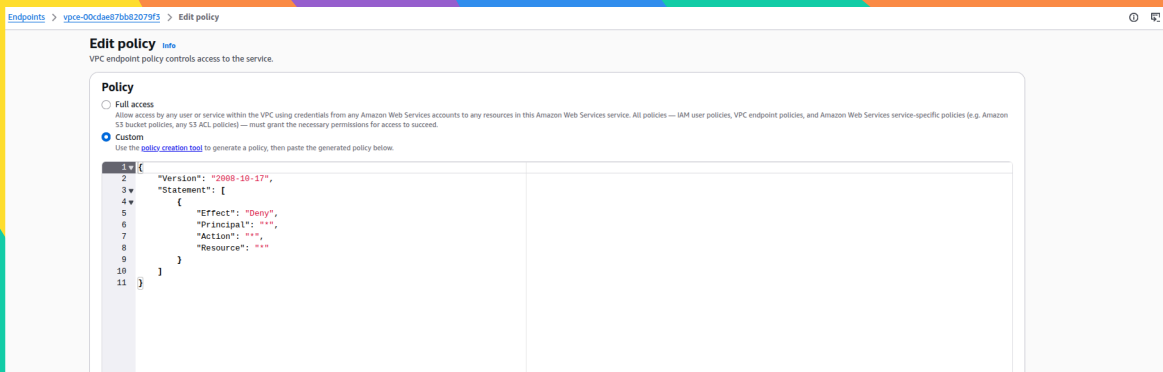




Endpoint policies

An endpoint policy is a set of rules that controls what users or services can access through a VPC endpoint.

I updated my endpoint's policy by changing the effect to deny. I could see the effect of this right away, because when I ran the command to list the object in the S3 bucket, there was an error.





nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

